Pagination in Express.js using **mongoose-paginate-v2**

Overview

Pagination is the process of splitting a large dataset into smaller parts (pages). It avoids sending thousands of records to the client at once, reducing server load and improving response time. When implementing pagination in an Express.js + MongoDB application, the **mongoose-paginate-v2** plugin provides a convenient, ready-to-use solution.

---

1. Why Pagination is Needed

Key Problems With Large Result Sets

- Sending large sets (500–1,000+ records) increases server memory usage.
- Slower response time when multiple users access data simultaneously.
- Users cannot read thousands of records at once; segmented views are more usable.

Pagination Example

If total records = 7 and we show 3 at a time:

- Page 1 → records 1–3
- Page 2 → records 4–6
- Page 3 → record 7

---

2. Setting Up Pagination with **mongoose-paginate-v2**

Step 1: Install the Package

```
npm install mongoose-paginate-v2
```

Step 2: Include Plugin in the Schema File

In your Mongoose schema file:

```
import mongoose from 'mongoose';
import mongoosePaginate from 'mongoose-paginate-v2';

const ContactSchema = new mongoose.Schema({
  name: String,
  email: String,
  phone: String
```

```
});

// Apply the plugin
ContactSchema.plugin(mongoosePaginate);

// Create model
const Contact = mongoose.model("Contact", ContactSchema);

export default Contact;
```

---

3. Using Pagination in the Controller

Basic Structure

Instead of:

```
const contacts = await Contact.find();
```

Use:

```
const result = await Contact.paginate(query, options);
```

Step-by-Step Implementation

Step A: Read Query Parameters

```
let { page, limit } = req.query;

page = parseInt(page) || 1;       // default: page 1
limit = parseInt(limit) || 3;     // default: show 3 records per page
```

Step B: Create Pagination Options

```
const options = {
  page,
  limit
};
```

Step C: Execute Pagination

```
const result = await Contact.paginate({}, options);
```

Step D: Data Returned by `paginate()`

`result` contains:

- `docs` → array of fetched records
- `totalDocs`
- `limit`
- `page` (current page)
- `hasPrevPage`
- `hasNextPage`
- `prevPage`
- `nextPage`
- `totalPages`
- `pagingCounter` (starting index number for the current page)

Step E: Send Data to View

```
res.render("home", {
  contacts: result.docs,
  totalPages: result.totalPages,
  currentPage: result.page,
  hasPrevPage: result.hasPrevPage,
  hasNextPage: result.hasNextPage,
  prevPage: result.prevPage,
  nextPage: result.nextPage,
  counter: result.pagingCounter
});
```

---

4. Creating Dynamic Pagination UI (Bootstrap)

Step 1: Base Bootstrap Pagination Markup

Copy from getbootstrap.com:

```
<nav>
  <ul class="pagination justify-content-center">
    <li class="page-item disabled"><a class="page-link">Previous</a></li>
    <li class="page-item"><a class="page-link">1</a></li>
    <li class="page-item"><a class="page-link">Next</a></li>
  </ul>
</nav>
```

5. Converting Pagination to Dynamic EJS

A. Generate Page Number Buttons

```
<% for (let i = 1; i <= totalPages; i++) { %>
  <li class="page-item <%= currentPage === i ? 'active' : '' %>">
    <a class="page-link" href="/?page=<%= i %>"><%= i %></a>
  </li>
<% } %>
```

B. Dynamic "Previous" Button

```
<li class="page-item <%= hasPrevPage ? '' : 'disabled' %>">
  <a class="page-link" href="/?page=<%= prevPage %>">Previous</a>
</li>
```

C. Dynamic "Next" Button

```
<li class="page-item <%= hasNextPage ? '' : 'disabled' %>">
  <a class="page-link" href="/?page=<%= nextPage %>">Next</a>
</li>
```

6. Displaying Sequential Row Numbers Correctly

In your table:

```
<td><%= counter + index %></td>
```

Where:

- `counter` is `pagingCounter` from the plugin
- `index` is the loop index starting from 0

This ensures sequences like:

- Page 1 → 1, 2, 3
- Page 2 → 4, 5, 6
- Page 3 → 7, 8

7. Changing Pagination Limit Dynamically

Just modify the `limit` value in the controller:

```
limit = parseInt(req.query.limit) || 5;
```

8. Summary of Key Concepts

What You Learned

1. Why pagination is important.
2. Installing and configuring **mongoose-paginate-v2**.
3. Writing controllers to paginate MongoDB data.
4. Rendering paginated data in EJS.
5. Creating dynamic Bootstrap pagination:
    - Page buttons
    - Previous/Next links
    - Active state styling
6. Maintaining correct serial numbers on each page.

Pagination Benefits

- Reduces server processing load.
- Improves performance for large datasets.
- Better user experience with manageable record chunks.